

Spam_Email_Detection_ML

Spam Email Detection Using Machine Learning This project aims to classify emails as Spam or legitimate using machine learning algorithms.

```
# Import required libraries
import pandas as pd
import numpy as np

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
# Import machine learning libraries

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
    confusion_matrix
)

print("Machine learning libraries imported successfully!")
```

Machine learning libraries imported successfully!

```
# Download the spam email dataset

import urllib.request

url = "https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/spam_dataset.tsv"
urllib.request.urlretrieve(url, "spam_dataset.tsv")

print("Dataset downloaded successfully!")
```

Dataset downloaded successfully!

```
# Load the dataset

df = pd.read_csv(
    "spam_dataset.tsv",
    sep="\t",
    header=None,
```

```

    names=["label", "message"]
)

print("Dataset loaded successfully!")
print("Number of rows:", len(df))
print("\nFirst 5 rows:")
display(df.head())

```

Dataset loaded successfully!
Number of rows: 5572

First 5 rows:

	label	message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

```

# Check for missing values and duplicate messages

print("Missing values:")
print(df.isnull().sum())

print("\nDuplicate rows:", df.duplicated().sum())

# Remove duplicates
df = df.drop_duplicates().reset_index(drop=True)

print("\nRows after removing duplicates:", len(df))

```

Missing values:
label 0
message 0
dtype: int64

Duplicate rows: 403

Rows after removing duplicates: 5169

```

# Separate features and target

```

```
X = df["message"]
y = df["label"]

# Convert labels: ham = 0, spam = 1
y = y.map({"ham": 0, "spam": 1})

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Training samples: 4135
Testing samples: 1034

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Create TF-IDF vectorizer
tfidf = TfidfVectorizer(
    lowercase=True,
    stop_words="english",
    max_features=5000
)

# Fit TF-IDF on training data and transform both datasets
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print("Training TF-IDF shape:", X_train_tfidf.shape)
print("Testing TF-IDF shape:", X_test_tfidf.shape)
```

Training TF-IDF shape: (4135, 5000)
Testing TF-IDF shape: (1034, 5000)

```
from sklearn.linear_model import LogisticRegression

# Create Logistic Regression model
logistic_model = LogisticRegression(max_iter=1000)

# Train the model
logistic_model.fit(X_train_tfidf, y_train)

# Make predictions
y_pred_lr = logistic_model.predict(X_test_tfidf)

print("Logistic Regression model trained successfully!")
```

Logistic Regression model trained successfully!

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
```

```

        recall_score,
        f1_score,
        roc_auc_score
    )

# Calculate evaluation metrics
accuracy_lr = accuracy_score(y_test, y_pred_lr)
precision_lr = precision_score(y_test, y_pred_lr)
recall_lr = recall_score(y_test, y_pred_lr)
f1_lr = f1_score(y_test, y_pred_lr)

# Probability for ROC-AUC
y_prob_lr = logistic_model.predict_proba(X_test_tfidf)[:, 1]
roc_auc_lr = roc_auc_score(y_test, y_prob_lr)

print("Logistic Regression Results")
print("-----")
print("Accuracy :", accuracy_lr)
print("Precision:", precision_lr)
print("Recall   :", recall_lr)
print("F1-score :", f1_lr)
print("ROC-AUC  :", roc_auc_lr)

```

```

Logistic Regression Results
-----
Accuracy : 0.9584139264990329
Precision: 0.9782608695652174
Recall   : 0.6870229007633588
F1-score : 0.8071748878923767
ROC-AUC  : 0.9915210536549077

```

```

from sklearn.ensemble import RandomForestClassifier

# Create Random Forest model
random_forest = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Train the model
random_forest.fit(X_train_tfidf, y_train)

# Make predictions
y_pred_rf = random_forest.predict(X_test_tfidf)

print("Random Forest model trained successfully!")

```

```

Random Forest model trained successfully!

```

```

# Calculate Random Forest metrics

accuracy_rf = accuracy_score(y_test, y_pred_rf)
precision_rf = precision_score(y_test, y_pred_rf)
recall_rf = recall_score(y_test, y_pred_rf)
f1_rf = f1_score(y_test, y_pred_rf)

# Probability for ROC-AUC
y_prob_rf = random_forest.predict_proba(X_test_tfidf)[:, 1]

```

```
roc_auc_rf = roc_auc_score(y_test, y_prob_rf)
```

```
print("Random Forest Results")
print("-----")
print("Accuracy :", accuracy_rf)
print("Precision:", precision_rf)
print("Recall   :", recall_rf)
print("F1-score :", f1_rf)
print("ROC-AUC  :", roc_auc_rf)
```

Random Forest Results

Accuracy : 0.9758220502901354
Precision: 0.9732142857142857
Recall : 0.8320610687022901
F1-score : 0.897119341563786
ROC-AUC : 0.9943234172774382

```
import pandas as pd
```

```
results = pd.DataFrame({
    "Model": ["Logistic Regression", "Random Forest"],
    "Accuracy": [accuracy_lr, accuracy_rf],
    "Precision": [precision_lr, precision_rf],
    "Recall": [recall_lr, recall_rf],
    "F1-score": [f1_lr, f1_rf],
    "ROC-AUC": [roc_auc_lr, roc_auc_rf]
})
```

```
print(results)
```

	Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
0	Logistic Regression	0.958414	0.978261	0.687023	0.807175	0.991521
1	Random Forest	0.975822	0.973214	0.832061	0.897119	0.994323

```
import matplotlib.pyplot as plt
```

```
metrics = ["Accuracy", "Precision", "Recall", "F1-score", "ROC-AUC"]
```

```
x = range(len(metrics))
```

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(x, results.loc[0, metrics], marker="o", label="Logistic Regression")
```

```
plt.plot(x, results.loc[1, metrics], marker="o", label="Random Forest")
```

```
plt.xticks(x, metrics)
```

```
plt.ylim(0, 1.05)
```

```
plt.xlabel("Evaluation Metrics")
```

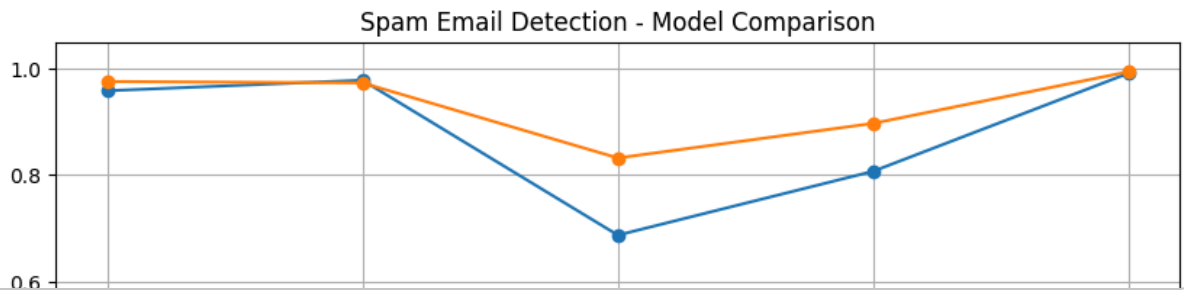
```
plt.ylabel("Score")
```

```
plt.title("Spam Email Detection - Model Comparison")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```



```
# Test the spam detection model with new messages

new_messages = [
    "Congratulations! You have won a free prize. Click here to claim now.",
    "Hi, are you coming to college tomorrow?"
]

# Convert messages into TF-IDF
new_messages_tfidf = tfidf.transform(new_messages)

# Predict using Logistic Regression
predictions = logistic_model.predict(new_messages_tfidf)

for message, prediction in zip(new_messages, predictions):
    if prediction == 1:
        print("Message:", message)
        print("Prediction: SPAM")
    else:
        print("Message:", message)
        print("Prediction: HAM")
    print("-----")
```

```
Message: Congratulations! You have won a free prize. Click here to claim
Prediction: SPAM
-----
Message: Hi, are you coming to college tomorrow?
Prediction: HAM
-----
```

```
# Final Model Comparison

print("FINAL MODEL COMPARISON")
print("=====")

print("\nLogistic Regression:")
print("Accuracy :", round(accuracy_lr, 4))
print("Precision:", round(precision_lr, 4))
print("Recall    :", round(recall_lr, 4))
print("F1-score  :", round(f1_lr, 4))
print("ROC-AUC   :", round(roc_auc_lr, 4))

print("\nRandom Forest:")
print("Accuracy :", round(accuracy_rf, 4))
print("Precision:", round(precision_rf, 4))
print("Recall    :", round(recall_rf, 4))
print("F1-score  :", round(f1_rf, 4))
print("ROC-AUC   :", round(roc_auc_rf, 4))
```

FINAL MODEL COMPARISON

=====

Logistic Regression:

Accuracy : 0.9584

Precision: 0.9783

Recall : 0.687

F1-score : 0.8072

ROC-AUC : 0.9915

Random Forest:

Accuracy : 0.9758

Precision: 0.9732

Recall : 0.8321

F1-score : 0.8971

ROC-AUC : 0.9943

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
```

```
# Confusion Matrix - Logistic Regression
cm_lr = confusion_matrix(y_test, y_pred_lr)
```

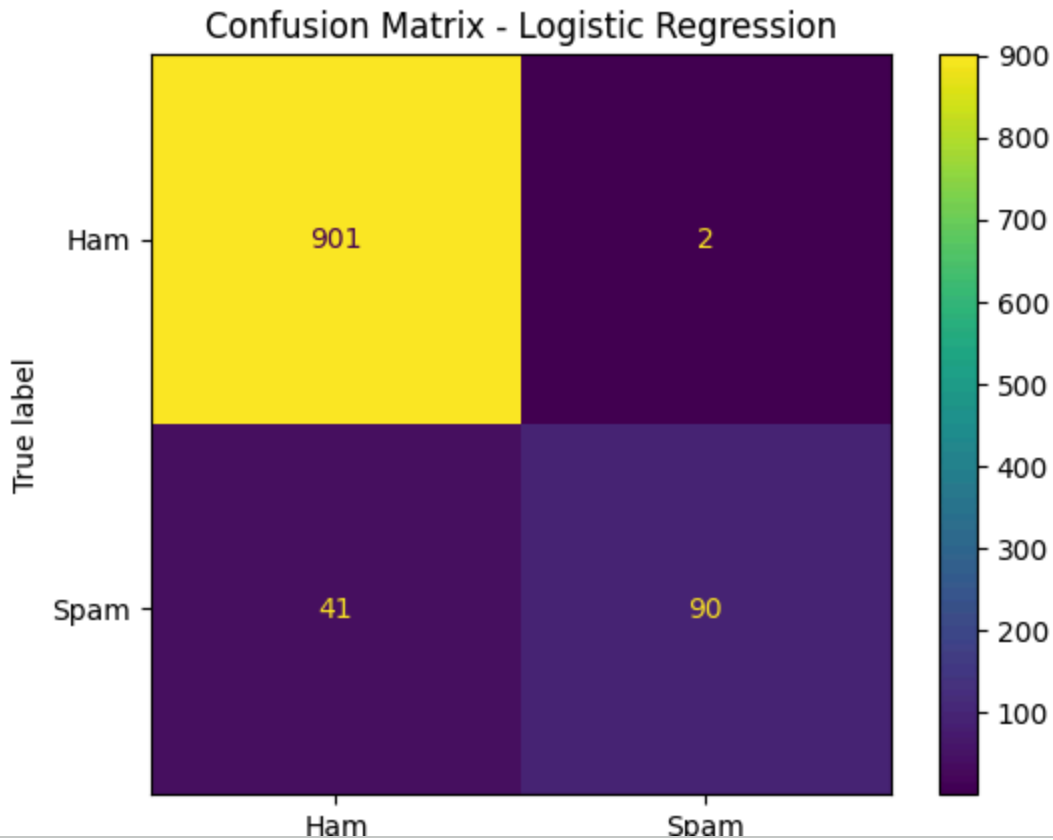
```
disp_lr = ConfusionMatrixDisplay(
    confusion_matrix=cm_lr,
    display_labels=["Ham", "Spam"]
)
```

```
disp_lr.plot()
plt.title("Confusion Matrix - Logistic Regression")
plt.show()
```

```
# Confusion Matrix - Random Forest
cm_rf = confusion_matrix(y_test, y_pred_rf)
```

```
disp_rf = ConfusionMatrixDisplay(
    confusion_matrix=cm_rf,
    display_labels=["Ham", "Spam"]
)
```

```
disp_rf.plot()
plt.title("Confusion Matrix - Random Forest")
plt.show()
```



```
# Test the spam detection model with new messages

def predict_spam(message):
    message_tfidf = tfidf.transform([message])
    prediction = random_forest.predict(message_tfidf)[0]

    if prediction == 1:
        return "Spam"
    else:
        return "Ham"

# Test messages
messages = [
    "Congratulations! You have won a free prize. Click now!",
    "Hi, are you coming to college tomorrow?",
    "You have won 1000 dollars. Claim your reward now!",
    "Can you send me the notes?"
]

for message in messages:
    print("Message:", message)
    print("Prediction:", predict_spam(message))
    print()
```

Message: Congratulations! You have won a free prize. Click now!
Prediction: Ham

Message: Hi, are you coming to college tomorrow?
Prediction: Ham

Message: You have won 1000 dollars. Claim your reward now!
Prediction: Spam

Message: Can you send me the notes?
Prediction: Ham

```
# Test the model with new messages

new_messages = [
    "Congratulations! You won a free lottery. Click now!",
    "Hey, are we meeting for college tomorrow?",
    "You have won a cash prize. Claim it immediately!",
    "Please send me the assignment notes."
]

# Convert messages into TF-IDF features
new_messages_tfidf = tfidf.transform(new_messages)

# Predict using Random Forest
predictions = random_forest.predict(new_messages_tfidf)

# Display results
for message, prediction in zip(new_messages, predictions):
    if prediction == 1:
        result = "SPAM"
    else:
        result = "HAM"

    print("Message:", message)
    print("Prediction:", result)
    print("-" * 50)
```

```
Message: Congratulations! You won a free lottery. Click now!
Prediction: HAM
-----
Message: Hey, are we meeting for college tomorrow?
Prediction: HAM
-----
Message: You have won a cash prize. Claim it immediately!
Prediction: SPAM
-----
Message: Please send me the assignment notes.
Prediction: HAM
-----
```

```
import joblib

# Save the TF-IDF vectorizer
joblib.dump(tfidf, "tfidf_vectorizer.pkl")

# Save Logistic Regression model
joblib.dump(logistic_model, "logistic_regression_model.pkl")

# Save Random Forest model
joblib.dump(random_forest, "random_forest_model.pkl")

print("All models and vectorizer saved successfully!")
```

All models and vectorizer saved successfully!

```
import joblib

joblib.dump(tfidf, "tfidf_vectorizer.pkl")
joblib.dump(logistic_model, "logistic_regression_model.pkl")
joblib.dump(random_forest, "random_forest_model.pkl")

print("Models saved successfully!")
```

Models saved successfully!

Conclusion: The Spam Email Detection system was successfully developed using Machine Learning. TF-IDF was used to convert text messages into numerical. Logistic Regression and Random Forest Models were trained and evaluated. Both models successfully classified new messages as spam or ham. The system achieved good performance on the test dataset and can be used to automatically identify spam messages.